

GRIFFON

B — Grow community contributions

Evidence document

Open-source modular security project for Linux
Rust • Tauri • CLI • Daemon • Plugins

Main repository: github.com/GriffonAV/griffon

Website and documentation: griffon-av.vercel.app

Evidence version — community exchanges and public resources included

1. Purpose of this document

This document brings together evidence of Griffon's efforts to open the project to contributions from the technical community. It is based only on verifiable elements: a public repository, an open-source license, contribution documentation, external discussions, and technical publications.

Summary: Griffon already has a credible open-source foundation and several documented exchanges with external communities. However, no merged external pull request has yet been confirmed. This remains the main missing element for fully satisfying the requirement.

Evidence currently available

- Public GitHub repository and a fully open-source project.
- Apache License 2.0 provided in a LICENSE file.
- CONTRIBUTING.md welcoming feedback, bug reports, and pull requests.
- Discussions with people outside the project team on Discord and Reddit.
- External technical advice taken into account for Linux packaging and inter-process communication.
- Public website with documentation and a technical blog.
- Technical article shared on daily.dev,

Evidence still to strengthen

- Complete the installation and reproduction instructions in CONTRIBUTING.md.
- Create issues with the good first issue and help wanted labels.
- Receive and merge at least one concrete external contribution, even a minor one.
- Collect a testimonial from an external user or contributor.

2. A public project with a clear license

2.1 Public GitHub repository

Griffon's main repository is publicly available on GitHub: <https://github.com/GriffonAV/griffon>. This allows anyone to inspect the source code, understand the architecture, open an issue, or propose a change.

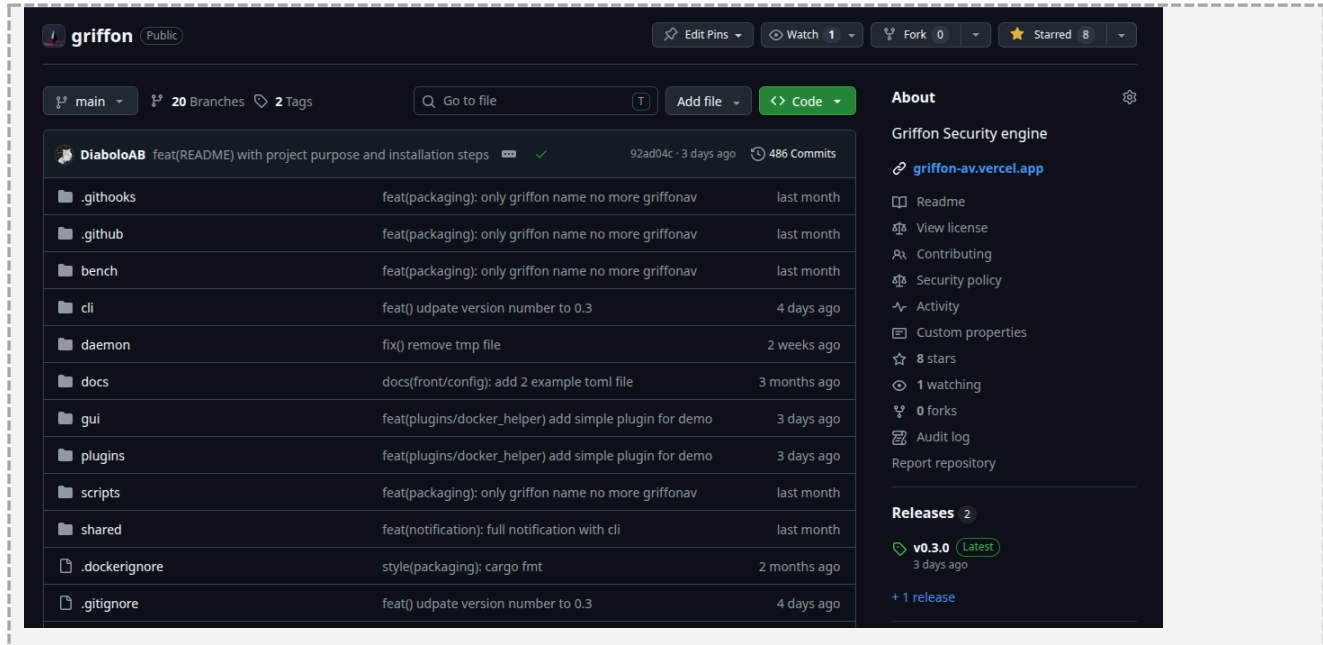


Figure 1 — Griffon's public GitHub repository.

2.2 Apache License 2.0

Griffon is distributed under the Apache License, Version 2.0, provided in a file named LICENSE at the root of the repository.

This license is consistent with the project's goals. It permits use, modification, and redistribution of the source code, including in professional contexts, provided that the license and attribution notices are preserved. It also contains explicit patent provisions and a limitation of liability suitable for an open-source project.

Rationale: Apache License 2.0 provides a permissive and professional framework while preserving clear attribution and explicit legal protections.



Figure 2 — LICENSE file confirming the use of Apache License 2.0.

2.3 Scope open to contributions

The entire project is open source and open to contributions, including:

- the Rust daemon;
- the command-line interface;
- the Vite/Tauri graphical interface;
- the plugin system;
- the documentation, tests, and packaging tools.

No functional part of the project is declared closed. Proposals must nevertheless be discussed and reviewed to remain compatible with the project's technical objectives and security requirements.

3. Documentation for contributors

3.1 CONTRIBUTING.md

The repository contains a CONTRIBUTING.md file that thanks prospective contributors and states that Griffon welcomes feedback, bug reports, and pull requests. It also lists the information expected in a bug report: operating system, Griffon version, reproduction steps, logs, and error messages.

For enhancements, contributors are invited to open an issue before writing code, particularly for the CLI, daemon, and GUI. This makes it possible to discuss the idea and confirm that it aligns with the project's current milestones.

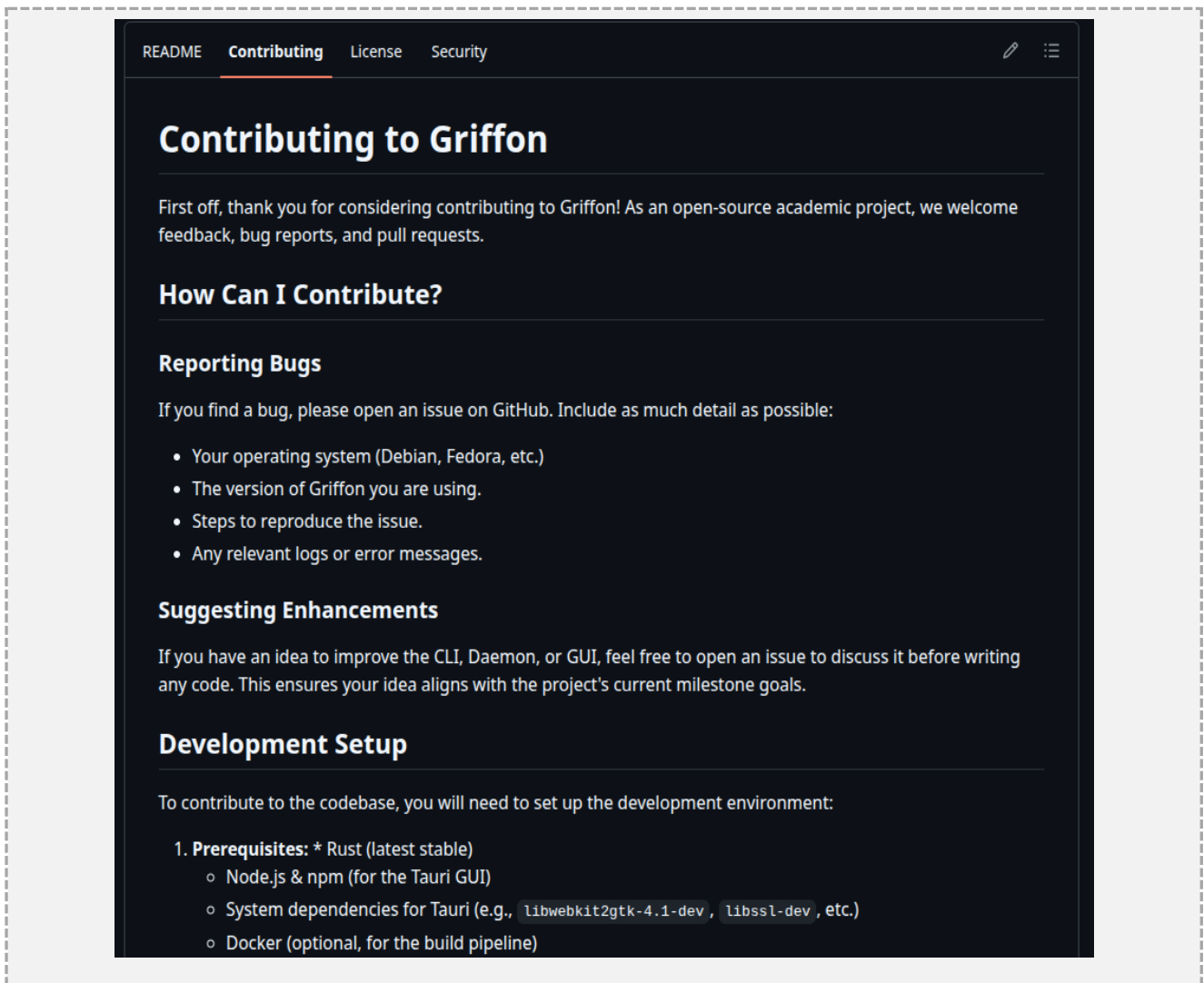


Figure 3 — Extract from CONTRIBUTING.md.

3.2 Documented development environment

The file already lists the main prerequisites: the latest stable Rust toolchain, Node.js and npm for the Tauri interface, Tauri system dependencies, and optionally Docker for the build pipeline.

Identified limitation: the setup section still contains a TODO. The file is present and useful, but reproduction commands must be completed so that a new contributor can start without direct assistance.

The main instructions still to be added are:

- clone the repository and install its dependencies;
- run `npm install` in the `gui` directory on first setup;
- start the application in development mode;
- run the tests, `cargo clippy`, and the TypeScript checks;
- explain the plugin workflow and the paths used in development.

4. Calls for advice and exchanges with technical communities

The team did more than publish the source code. It actively requested advice from external communities on concrete technical choices. These exchanges took place on a Rust Discord server, Reddit, and `daily.dev`.

4.1 Discord discussion about IPC security

On a Rust-focused Discord server, in a security channel, the team asked for technical advice on securing communication between Griffon processes, especially between the GUI and the privileged daemon.

Community members explained that most POSIX IPC mechanisms rely on operating-system permissions, suggested socket pairs or pipes when one process spawns another, and highlighted `SO_PEERCRED` and `SCM_CREDENTIALS` as mechanisms for checking the identity of the peer connected to a Unix socket. These exchanges helped the team define the security requirements and guide the IPC architecture used by the project.

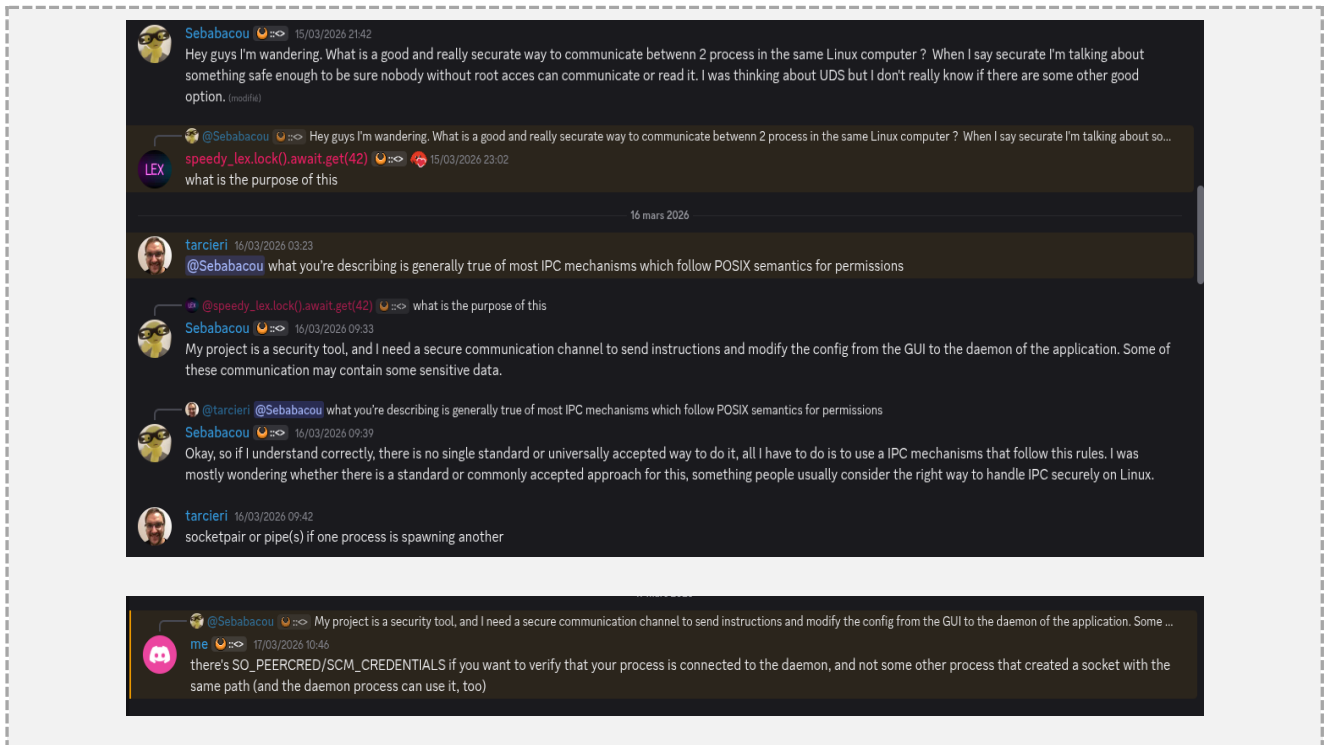


Figure 4 — Discussion with the Rust community about secure inter-process communication.

4.2 Reddit post about Linux packaging

A post was published on Reddit to ask how to package a Linux application made up of several components: a Rust daemon, a CLI, a Vite/Tauri GUI, and plugins. The team wanted to avoid choosing a tool that was unsuitable for a multi-binary workspace and a real system service.

External replies recommended treating the application like a Linux distribution package: installing executables in `/usr/bin`, plugins in `/usr/lib`, and resources in `/usr/share`. They also distinguished self-contained formats such as Flatpak, Snap, or Apptainer from native distribution packages that are more appropriate for a daemon running as a system service.

Following this discussion, Griffon selected nFPM to generate Linux packages, notably in `.deb` and `.rpm` formats. The Reddit exchange therefore provides evidence of external advice leading to a concrete technical decision.

 **r/linuxquestions** • 4mo ago
Diabolo__

Recommendation to package my app.

Hi everyone! I've been doing some research, but I'm still not sure what the best approach is, as this is my first time packaging a Linux app.

My friends and I are building a project divided into several parts: a Rust daemon, a CLI, and a GUI using Vite and Tauri. The app also loads plugins, and we need a standard way to bundle some default ones. I know cargo-deb exists, but since our application has so many moving parts, I'm worried it might be the wrong tool for a multi-binary workspace.

Does anyone have recommendations for tools or resources for packaging 'suites' like this? Thanks in advance!

↑ 4 ↓ · 5

 **ThePowerOfPinkChicks** • 4mo ago

If your daemon is supposed to be a real system service (e.g., starts at boot, shared by multiple users, integrates with system logging), aim for a distro package and accept the extra `debian/ / rpm` work. If the service is mostly an internal helper for the GUI and you care more about "download and run anywhere," favor Flatpak/Snap/AppImage and let Tauri's bundling plus the sandbox manifest do most of the bundling for you. The one big missing piece for a precise recommendation is your distribution target: are you primarily aiming at Debian/Ubuntu-style repos, or do you just want easy cross-distro downloads?

⊖ ↑ 4 ↓

 **Diabolo__** OP • 4mo ago

I want it to be cross-distro, sorry I forgot to precise this. Yes our daemon is a real system service. We were already thinking about building a distro package. The question was more about how to make the package itself, I have no idea how it's working. Is it just a bunch of binary organized in a specific way, with some installation script ?

Thank you for the help :D

⊖ ↑ 4 ↓

 **RevolutionaryHigh** • 4mo ago

rpmbuild for rpm, for .deb I don't know, google yhow to make deb package

⊖ ↑ 0 ↓

 **MintAlone** • 4mo ago

building a deb package is relatively simple:

```
dpkg-deb --build /folder-containing-files
```

The folder is treated as `/` and files in there are copied to same location in the host filesystem. There is also a DEBIAN folder with a control file and optional pre and post install scripts. Not difficult for a simple application.

Don't know rpm, I'm a debian/ubuntu user.

↑ 2 ↓

Figure 5 — Reddit request and external replies about packaging Griffon for Linux.

4.3 Communities reached

The project was discussed on r/rust and r/linuxquestions, as well as in a Rust Discord security channel. These communities match the project's technical scope: Rust development, Linux integration, system services, packaging, and security.

5. Technical outreach: website, documentation, blog, and daily.dev

5.1 Official website and documentation

Griffon has a public website at <https://griffon-av.vercel.app/>. It brings together the project overview, installation links, documentation, and technical blog. This gives users and prospective contributors a clear entry point without requiring them to browse the entire source tree first.



Figure 6 — Griffon's official public website.

5.2 Technical blog

The blog presents analyses, experiments, and decisions made during development. It makes the project's technical reasoning more transparent and shares research results with people interested in the project's challenges.

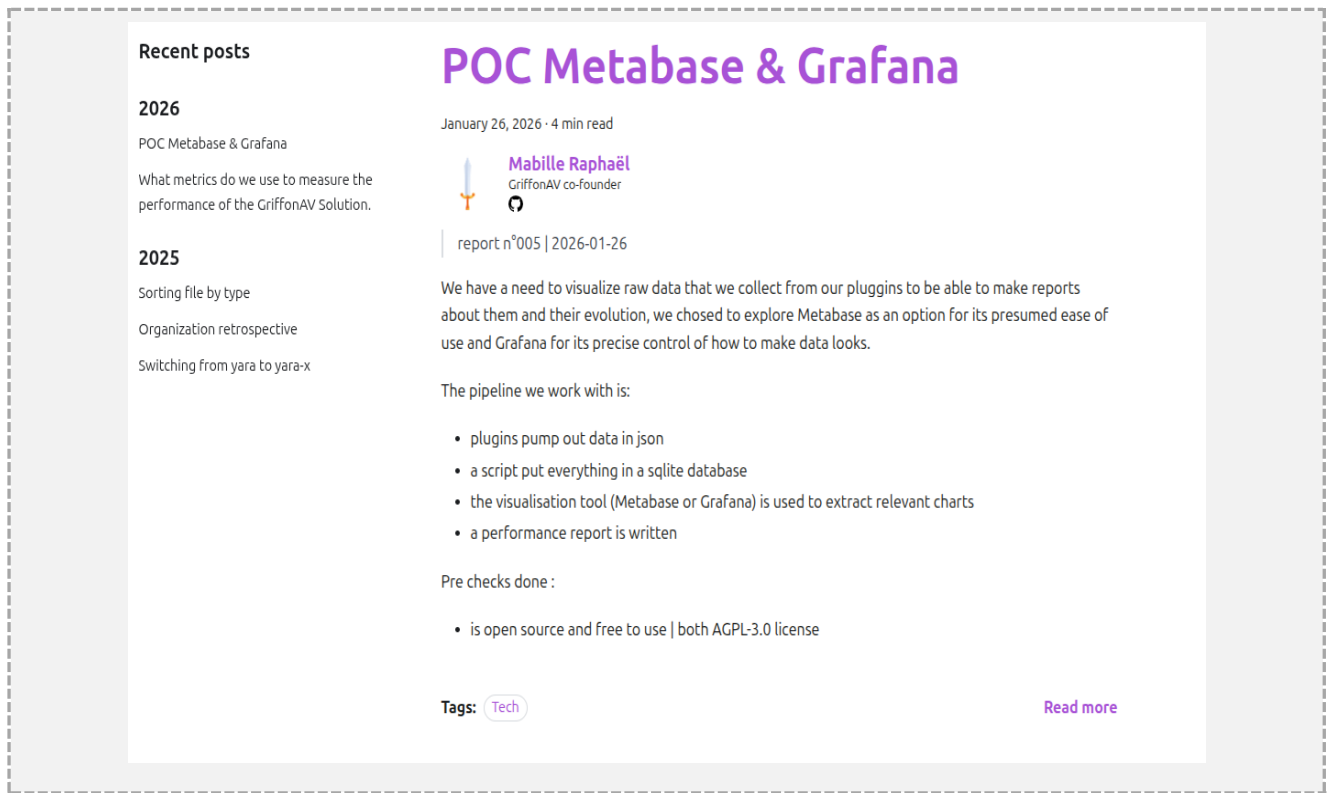


Figure 7 — Griffon’s public technical blog.

5.3 Publication on daily.dev

The article “Griffon AV | Data Visualisation | Metabase VS Grafana” was shared on daily.dev. It introduces Griffon as a fast, modular, and secure Linux security engine written in Rust and links readers to the project blog for the full comparison of the two data-visualisation tools.

The post received 10 reactions, providing evidence that the project was shared beyond its own GitHub repository and reached an external technical audience.

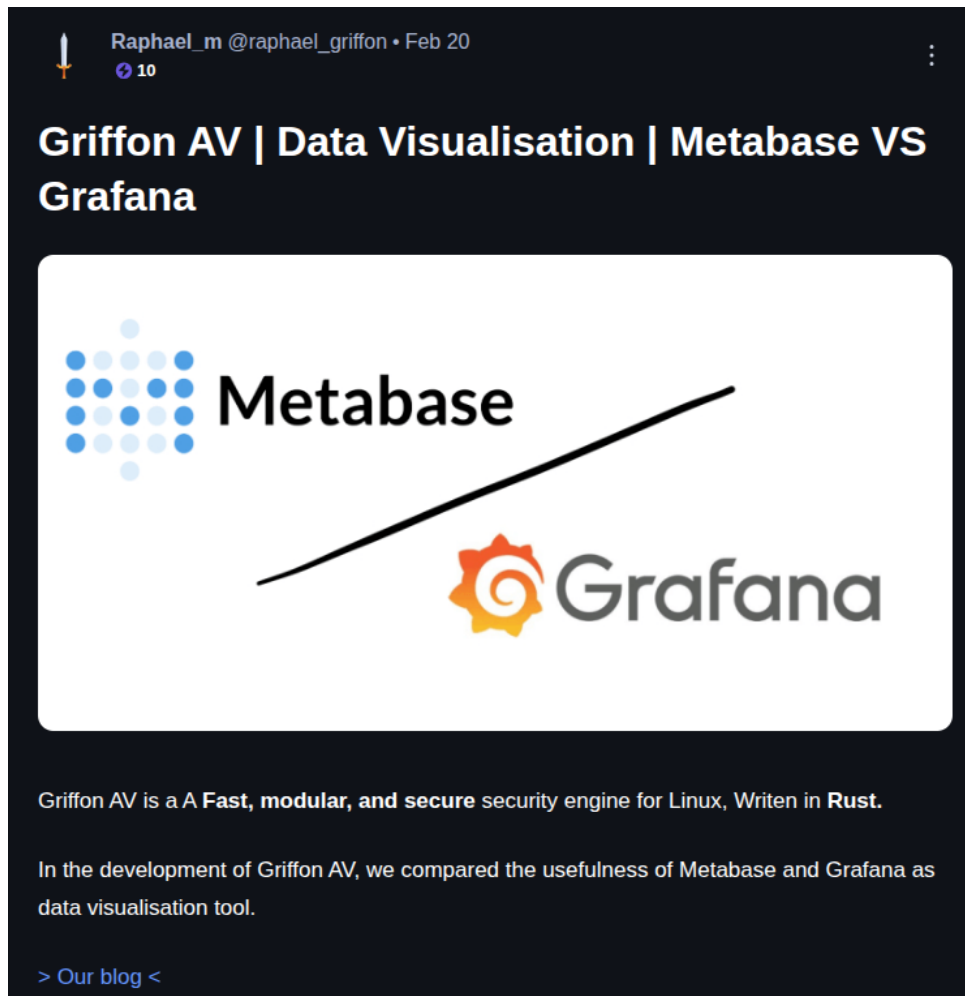


Figure 8 — daily.dev publication about data visualisation, showing 10 reactions.

Post: daily.dev/posts/griffon-av-data-visualisation-metabase-vs-grafana-0ssnep1gv

6. External feedback translated into project decisions

The community exchanges had a concrete effect on the project. Two examples are particularly relevant:

Topic	External input	Decision or effect on Griffon
Linux packaging	Advice on native distribution packages, file layout, and the limits of Tauri bundling.	Selection of nFPM to build .deb and .rpm packages for a multi-binary suite.
IPC communication	Rust community discussion about secure communication and peer verification between local processes.	The exchanges helped define the security requirements and guide the project's IPC architecture.

Traceability: the Reddit and Discord screenshots include the community context and the replies received. They connect external advice with the decisions made by the project team.

7. Current state of external contributions

People outside the Griffon team have already commented on project ideas through Reddit and Discord. These exchanges constitute external community feedback and influenced technical choices.

However, at the time of writing, no external contribution in the form of a merged pull request, commit, or documentation correction has been confirmed. No formal testimonial or confirmed external user test has been collected either.

8. Alignment with the expected deliverables

Expected item	Available evidence	Status
Public repository	Public GriffonAV/griffon GitHub repository	VALIDATED
Open-source license	LICENSE file — Apache 2.0	VALIDATED
README and documentation	README, public website, documentation, and blog	VALIDATED
CONTRIBUTING.md	File present with bug-report and enhancement guidance	PARTIAL
Installation/reproduction instructions	Prerequisites are present, but detailed setup remains a TODO	TO COMPLETE
Call to the community	Reddit, Discord, and daily.dev publications and discussions	VALIDATED
Feedback tracking	Screenshots of exchanges linked to resulting technical decisions	VALIDATED

Expected item	Available evidence	Status
Integrated external contribution	External comments received, but no confirmed merged PR	PARTIAL
Community visibility	Website, blog, daily.dev, and relevant technical communities	VALIDATED

9. Conclusion

Griffon is more than a repository made public. The project has an explicit open-source license, contribution guidelines, a documentation website, and a technical blog. The team also reached out to external technical communities and used their feedback to guide concrete decisions, particularly Linux packaging with nFPM and the security requirements for IPC between processes.

The available evidence demonstrates a genuine effort to open the project and engage with the community. The main remaining weakness is the absence of a formally integrated external contribution. A small, properly tracked and acknowledged pull request would complete the evidence and cover the remaining expectation.

Assessment: community engagement is active and documented; full validation still depends on integrating at least one external contribution.